



Laporan Praktikum Algoritma & Pemrograman

Semester Genap 2025/2026

SAYA MENYATAKAN BAHWA LAPORAN PRAKTIKUM INI SAYA BUAT DENGAN USAHA SENDIRI TANPA MENGGUNAKAN BANTUAN ORANG LAIN. SEMUA MATERI YANG SAYA AMBIL DARI SUMBER LAIN SUDAH SAYA CANTUMKAN SUMBERNYA DAN TELAH SAYA TULIS ULANG DENGAN BAHASA SAYA SENDIRI.

SAYA SANGGUP MENERIMA SANKSI JIKA MELAKUKAN KEGIATAN PLAGIASI, TERMASUK SANKSI TIDAK LULUS MATA KULIAH INI.

NIM	71251214
Nama Lengkap	Raffiel Gideon Nyolo Nyolo
Minggu ke / Materi	13 / Tipe Data Tuples

PROGRAM STUDI INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS KRISTEN DUTA WACANA
YOGYAKARTA
2026

BAGIAN 1: MATERI MINGGU INI (40%)

Pada bagian ini, tuliskan kembali semua materi yang telah anda pelajari minggu ini. Sesuaikan penjelasan anda dengan urutan materi yang telah diberikan di saat praktikum. Penjelasan anda harus dilengkapi dengan contoh, gambar/ilustrasi, contoh program (source code) dan outputnya. Idealnya sekitar 5-6 halaman.

A. Tuple Immutable

Tuple dikatakan menyeruai list dan tidak bisa menggunakannya sebagai nama variabel. Nilai yang disimpan di dalam tipe data ini dapat berupa apapun yang diberikan indeks integer. Perbedaannya terletak pada sifat tuple yang immutable atau anggota di dalamnya tidak bisa diubah. Tuple dapat dicompare dan bersifat hashtable sehingga dapat dimasukkan dalam list sebagai key pada dictionary python.

Objek disebut hashable jika memiliki nilai hash yang tidak pernah berubah (method `__hash__()`), dan dapat dibandingkan dengan objek-objek lainnya (method `__eq__()` atau `__cm__()`). Objek hashtable membandingkan yang sama dengan memiliki nilai hash yang harus sama.

Penulisan sintaks tuple terdapat 3 macam, menggunakan tanda kurung, tidak menggunakan tanda kurung dan khusus untuk tuple hanya terdapat 1 elemen saja maka harus ditambahkan koma (,) dibelakang penulisan tuple. Jika tidak menggunakan koma, maka akan dianggap sebagai string. Berikut ini merupakan contoh penulisan tuple:

```
>>> t = 'a','b','c','d','e' | >>> t = ('a','b','c','d','e')
>>> t1 = ('a',)
>>> type(t1)
<type 'tuple'>
```

Terdapat model sintaks lain dengan menggunakan fungsi **built-in** tuple. Ketika tidak diisi dengan argument apapun, secara langsung akan membentuk tuple kosong (). Tetapi jika argumennya berupa urutan (string, list, atau tuple), akan mengembalikan nilai tuple dengan elemen-elemen yang berurutan. Contohnya:

```
t = tuple('aku kuat')

print (t)

('a', 'k', 'u', ' ', 'k', 'u', 'a', 't')
```

Sebagian besar operator yang bekerja pada list bekerja juga pada tuple. Tanda kurung kotak [] menandakan indeks element dari tuple. Untuk menampilkan rentang nilai dari element tuple dapat menggunakan metode slicing dan karena tuple bersifat immutable artinya element pada tuple tidak dapat berubah. Jika anda berusaha mengubah tuple, maka akan didapatkan error.

```
>>> t = ('a', 'b', 'c', 'd', 'e')
>>> print(t[2])
'c'

>>> t = ('a', 'b', 'c', 'd', 'e')
>>> print(t[2:4])
('b', 'd')

>>> t[0] = 'A'
TypeError: object doesn't support item assignment
```

Element pada tuple tidak dapat dirubah tetapi dapat diganti dengan elemen lain.

```
>>> t = ('A',) + t[1:]
>>> print(t)
('A', 'b', 'c', 'd', 'e')
```

B. Membandingkan Tuple

Operator compare dapat bekerja pada tuple. Cara kerjanya dengan membandingkan elemen pertama dari setiap sekuensial yang ada. Jika ada kesamaan , akan berlanjut ke elemen berikutnya. Proses ini terus meneruns berlanjut hingga ditemukan adanya perbedaan.

```
>>> (0, 1, 2) < (0, 3, 4)

True

>>> (0, 1, 5) < (0, 3, 4)

True
```

Fungsi (sort) pada python bekerja seperti biasanya. Tetapi pada kasus tertentu akan mengurutkan berdasarkan elemen kedua dan sebagainya. Fitur ini disebut dengan DSU (Decorate, Sort, Undercorate).

Decorate, merupakan urutan sekuensial yang membangun daftar tuple dengan satu atau lebih key pengurutan sebelum elemen pertama. **Sort**, merupakan list tuple yang menggunakan sort. **Undercorate**, melakukan ekstrasi pada elemen yang telah diurutkan pada satu sekuensial.

```
kalimat = 'aku anak sehat tubuhku kuat'
dalkata = kalimat.split()
t = list()
for kata in dalkata:
    t.append((len(kata), kata))

t.sort(reverse=True)

urutan = list()
for panjang, kata in t:
    urutan.append(kata)
print(urutan)
Output: ['tubuhku', 'sehat', 'kuat', 'anak', 'aku']
```

Looping yang pertama akan membuat daftar tuple, yang berisi daftar kata sesuai dengan panjangnya. Fungsi sort akan membandingkan elemen pertama dari list dari panjang kata yang ada dan kemudian akan menuju ke elemen kedua jika kondisi sesuai. Keyword reverse=True digunakan untuk melakukan urutan secara terbalik.

Looping kedua akan membangun daftar tuple dalam sesuai urutan alfabet diurutkan berdasarkan panjangnya. Pada contoh diatas Dua kata dalam kalimat akan diurutkan secara alfabet terbalik. Kata "kuat" akan muncul sebelum "anak" didalam list.

C. Penugasan Tuple

Salah satu fitur yang ada di Python adalah memiliki tuple disisi kiri dari statemen penugasan. Fitur ini mengijinkan untuk menetapkan lebih dari satu variabel pada sisi sebelah kiri secara berurutan. Tuple yang digunakan pada menggunakan statement penugasan disebelah kiri dapat di tulis menggunakan tanda kurung dan tidak menggunakan tanda kurung.

```
>>> m = [ 'punya', 'kamu' ]
>>> x, y = m
>>> x
'punya'
>>> y
'kamu'
```

```
>>> m = [ 'punya', 'kamu' ]
>>> (x, y) = m
>>> x
'punya'
```

Pada tuple bisa melakukan “menukar variabel” dalam satu statement. Perhatikan contoh berikut:

```
>>> a, b = b, a
```

Bagian kiri merupakan tuple dari variable dan bagian kanan merupakan tuple dari expresions. Tiap nilai pada bagian kanan diberikan/ditugaskan ke masing-masing variable di sebelah kiri. Semua expresions pada bagian kiri dievaluasi sebelum diberikan penugasan. Perlu di perhatikan jumlah variabel sisi kiri dan kanan harus sama.

Pada sisi sebelah kanan biasanya terdapat data sekuensial string, list atau tuple. Sebagai contoh, kita akan membagi alamat email menjadi user name dan domain seperti berikut ini.

```
email = 'cakodidi@ti.ukdw.ac.id'
nama, domain = email.split("@")
print ("Nama: " + nama, "Domain: " + domain)
Output: Nama: cakodidi Domain: ti.ukdw.ac.id
```

D. Dictionaries and Tuple

Dictionaries mempunyai metode yang disebut *items* untuk mengembalikan nilai daftar dari tuple yang masing-masing berisi *value* dan *key*

```
A = {'a': 13, 'b' : 7, 'c' : 20 }
d = list(A.items())
print (d)
Output: [('b', 7), ('a', 13), ('c', 20)]

A = {'a': 13, 'b' : 7, 'c' : 20 }
d = list(A.items())
d.sort()
print (d)
Output: [('a', 13), ('b', 7), ('c', 20)]
```

Nilai item yang dikembalikan tidak berurutan, seperti yang terlihat dalam dictionary biasa. Bagaimana pun juga, melakukan konversi di dictionary dari list tuple menampilkan isi dictionary yang diurutkan berdasarkan kuncinya, karena list tuple membandingkannya sehingga kita dapat melakukan sort pada tuple.

E. Multipenugasan dengan dictionaries

Kombinasi antara *items*, *tuple assignment* dan *for* akan menghasilkan kode tertentu pada *key* dan *value* dari *dictionary* dalam satu loop.

```
for key, val in list(d.items()):
    print(val, key)
```

Pada bagian looping ini, item mengembalikan list dari tuple, dan key. val adalah penugasan tuple yang melakukan iterasi berulang melalui pasangan key-value pada dictionary. Key dan value akan bergerak maju menuju pasangan key-value berikutnya di dictionary pada setiap iterasi loop. Mereka tetap dalam urutan hash.

Output:

```
10 a
22 c
1 b
```

Kita dapat mencetak isi kamus yang diurutkan berdasarkan nilai yang disimpan di setiap pasangan nilai kunci jika kedua metode di atas digabungkan.

Langkah pertama adalah membuat list tuple (value, key) dengan method [] agar pengurutan dilakukan berdasarkan value, bukan key. Setelah itu, list diurutkan secara terbalik dan dicetak. Menempatkan value sebagai elemen pertama tuple memudahkan pengurutan isi dictionary berdasarkan nilainya.

```
A = {'a': 13, 'b': 7, 'c': 20}
d = []
for key, val in A.items():
    d.append((val, key))
print(d)
Output: [(13, 'a'), (7, 'b'), (20, 'c')]
```

```
A = {'a': 13, 'b': 7, 'c': 20}
d = []
for key, val in A.items():
    d.append((val, key))
    d.sort(reverse=True)
print(d)
Output: [(20, 'c'), (13, 'a'), (7, 'b')]
```

F. Kata yang sering muncul

```
import string
fhand = open('romeo-full.txt')
counts = dict()
for line in fhand:
    line = line.translate(str.maketrans('', '', string.punctuation))
    line = line.lower()
    words = line.split()
    for word in words:
        if word not in counts:
            counts[word] = 1
        else:
            counts[word] += 1

lst = list()
for key, val in list(counts.items()):
    lst.append((val, key))
lst.sort(reverse=True)
for key, val in lst[:10]:
    print(key, val)
```

```
61 i
42 and
40 romeo
34 to
34 the
32 thou
32 juliet
30 that
29 my
24 thee
```

Program di atas digunakan untuk menghitung jumlah kemunculan setiap kata dalam file romeo-full.txt. Program akan menghapus tanda baca, mengubah semua huruf menjadi huruf kecil, lalu memisahkan teks menjadi kata-kata. Setelah itu, setiap kata dihitung frekuensi kemunculannya menggunakan dictionary. Data kemudian diubah menjadi list tuple (jumlah, kata), diurutkan dari yang terbesar ke terkecil, dan akhirnya menampilkan 10 kata yang paling sering muncul beserta jumlah kemunculannya.

G. Tuple sebagai kunci dictionaries

Tuple merupakan hashable dan list tidak. Ketika kita ingin membuat composite key yang digunakan dalam dictionary, kita dapat menggunakan tuple sebagai key.

Misalnya menggunakan composite key jika ingin membuat direktori telepon yang memetakan dari pasangan last-name, first-name ke nomor telepon. Dengan asumsi bahwa kita telah mendefinisikan variabel last, first, dan nomor. Penulisan pernyataan penugasan dalam dictionary sebagai berikut:

```
directory[last,first] = number
```

Expression yang ada didalam kurung kotak adalah tuple. Langkah selanjutnya dengan memberikan penugasan tuple pada looping for yang berhubungan dengan dictionary. Perhatikan contoh di bawah.

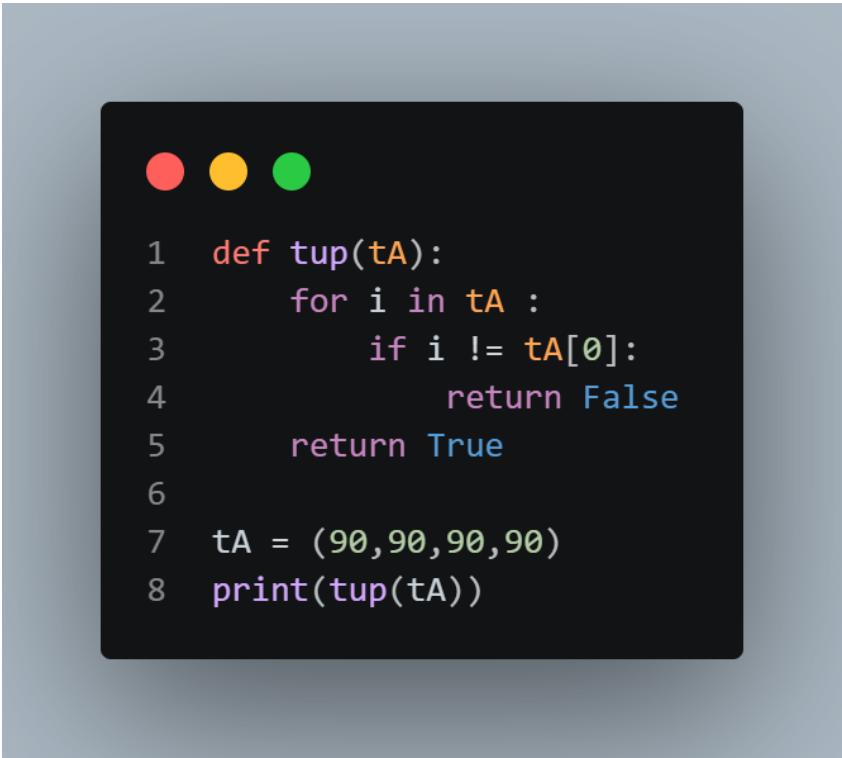
```
last = "Sucipto"
first = "Adi"
number = "088112266"
directory = dict()
directory[last, first] = number
for last, first in directory:
    print(first, last, directory[last,first])
```

Output: Adi Sucipto 088112266

BAGIAN 2: LATIHAN MANDIRI (60%)

Pada bagian ini anda menuliskan jawaban dari soal-soal Latihan Mandiri yang ada di modul praktikum. Jawaban anda harus disertai dengan source code, penjelasan dan screenshot output.

SOAL 1



```
1 def tup(tA):
2     for i in tA :
3         if i != tA[0]:
4             return False
5     return True
6
7 tA = (90,90,90,90)
8 print(tup(tA))
```

Penjelasan:

Program Python tersebut mendefinisikan sebuah fungsi bernama `tup(tA)` yang bertugas memeriksa apakah semua elemen dalam sebuah tuple sama dengan elemen pertamanya. Caranya, fungsi melakukan iterasi terhadap setiap elemen di dalam tuple, lalu membandingkannya dengan elemen pertama (`tA[0]`). Jika ditemukan ada elemen yang berbeda, fungsi langsung mengembalikan nilai `False`. Namun, jika seluruh elemen sama, maka setelah selesai iterasi fungsi akan mengembalikan `True`. Pada contoh yang diberikan, tuple `tA` berisi `(90, 90, 90, 90)` sehingga semua elemen identik, dan hasil dari `print(tup(tA))` adalah `True`.

Output:

```
PS D:\Semester_2\Praktikum Algoritma  
True
```

SOAL 2

A screenshot of a code editor with a dark background and light-colored text. The code is a Python function named 'bio' that takes a tuple 'data' as input. It prints the NIM, name, and address from the tuple. It also processes the name by splitting it into first and last names, converting the first name to lowercase, and reversing the last name. Finally, it calls the 'bio' function with a specific data tuple.

```
1 def bio(data):  
2     print("NIM :", data[1])  
3     print("NAMA :", data[0])  
4     print("ALAMAT :", data[2])  
5  
6     nim = tuple(data[1])  
7     print("NIM:", nim)  
8  
9     namaDepan = tuple(data[0].split()[0].lower())  
10    print("NAMA DEPAN:", namaDepan)  
11  
12    namaBelakang = data[0].split()[0:]  
13    namaBelakang.reverse()  
14    namaBelakang = tuple(namaBelakang)  
15    print("NAMA TERBALIK:", namaBelakang)  
16  
17    data = ("Raffiel Gideon Nyolo Nyolo", "71251214", "Buyumpondoli")  
18    bio(data)
```

Penjelasan:

Program Python ini mendefinisikan sebuah fungsi bernama `bio(data)` yang menerima sebuah tuple berisi informasi pribadi: nama, NIM, dan alamat. Di dalam fungsi, pertama-tama program mencetak NIM, nama, dan alamat sesuai urutan elemen tuple. Selanjutnya, NIM diubah menjadi tuple berisi karakter-karakter penyusunnya, lalu ditampilkan. Program juga mengambil nama depan (kata pertama dari nama lengkap), mengubahnya menjadi huruf kecil, dan menyimpannya sebagai tuple karakter. Setelah itu, nama lengkap dipecah menjadi daftar kata, dibalik urutannya dengan `reverse()`, lalu diubah menjadi tuple dan dicetak sebagai "NAMA TERBALIK". Pada bagian akhir, tuple data berisi ("Raffiel Gideon Nyolo Nyolo", "71251214", "Buyumpondoli") dipanggil ke fungsi `bio(data)`, sehingga seluruh proses di atas dijalankan dan hasilnya ditampilkan ke layar.

Output:

```
PS D:\Semester_2\Praktikum Algoritma dan Pemrograman\71
NIM : 71251214
NAMA : Raffiel Gideon Nyolo Nyolo
ALAMAT : Buyumpondoli
NIM: ('7', '1', '2', '5', '1', '2', '1', '4')
NAMA DEPAN: ('r', 'a', 'f', 'f', 'i', 'e', 'l')
NAMA TERBALIK: ('Nyolo', 'Nyolo', 'Gideon', 'Raffiel')
```

SOAL 3

A screenshot of a code editor with a dark background and light-colored text. The code is a Python script that takes a filename as input, attempts to open it, and processes its contents. It specifically looks for lines starting with 'From ' and extracts the time part (HH:MM:SS) to count its occurrences. The script uses a dictionary to store these counts and then sorts them by value. The code is as follows:

```
1  fname = input("Masukkan nama file : ")
2  try:
3      fhand = open(fname)
4  except:
5      print('file error tidak dapat dibuka:', fname)
6      exit()
7
8  counts = dict()
9  for line in fhand:
10     if line.startswith('From '):
11         words = line.split()
12         time = words[5].split(":")[0]
13         counts[time] = counts.get(time, 0) + 1
14
15  jam = sorted(counts.items())
16  for kunci, nilai in jam:
17     print(f"{kunci} {nilai}")
```

Penjelasan:

Program Python ini bekerja untuk menganalisis file teks, khususnya baris yang diawali dengan kata "From ", yang biasanya berisi informasi pengirim email beserta waktu pengiriman. Pertama, program meminta pengguna memasukkan nama file, lalu mencoba membukanya; jika gagal, akan muncul pesan error dan program berhenti. Setelah file berhasil dibuka, program membaca setiap baris dan hanya memproses baris yang sesuai. Dari baris tersebut, program mengambil elemen waktu (format HH:MM:SS), kemudian memisahkan bagian jam (HH) dan menyimpannya dalam dictionary sebagai kunci dengan nilai berupa jumlah kemunculan. Proses ini dilakukan berulang hingga seluruh baris selesai dibaca. Selanjutnya, dictionary diurutkan

berdasarkan jam, lalu hasilnya ditampilkan dalam bentuk pasangan jam dan jumlah kemunculan. Dengan demikian, program ini memberikan gambaran distribusi frekuensi email berdasarkan jam pengiriman, sehingga kita dapat mengetahui jam-jam tertentu yang paling sering muncul dalam data.

Output:

```
PS D:\Semester_2\Praktikum Algoritma
Masukkan nama file : mbox-short.txt
04 3
06 1
07 1
09 2
10 3
11 6
14 1
15 2
16 4
17 2
18 1
19 1
```